OCOM5300M Artificial Intelligence Project Vocabulary

In [1]:
```python
# Custom Classes
from captions import Captions
from vocabulary import Vocabulary

# Data Handling And Analysis
import pandas as pd  # data manipulation and analysis using DataFrame structures
import re # regular-expressions, i.e., for patern matching and text preprocessing

# Visualization
import matplotlib.pyplot as plt # plotting and visualizing results

# XML Processing
import xml.etree.ElementTree as ET

# Utilities
import os  # file and directory operations
```

In [2]:
```python
# suppress Intel specific Math Kernel Library (MKL) warnings
os.environ['KMP_DUPLICATE_LIB_OK'] = 'True'
```

In [3]:
```python
# display the complete content of DataFrame columns without truncation
pd.set_option('display.max_colwidth', None);
```

In [4]:
```python
# seed set to ensure reproducibility of results
seed = 42
```

Prepare Training Captions

In [5]:
```python
# to prevent data leakage the vocabulary is derived exclusively from the ROCOv2 training captions with 59958
records
training_captions_filename = "ROCOv2/train_captions.csv"
```

In [6]:
```python
# instantiate captions object
training_captions = Captions()
```

In [7]:
```
# load captions
training_caption_data = training_captions.load_captions(training_captions_filename)
```

In [8]:
```
# confirm number of training captions (records)
len(training_caption_data)
```

Out[8]:
```
59958
```

In [9]:
```
# clean captions
training_captions.clean_captions();
```

In [10]:
```
# confirm number of captions as 59958
assert len(training_caption_data) == 59_958
```

In [11]:
```
# display sample IDs and associated cleaned captions
training_caption_data.head()
```

Out[11]:

	ID	Caption	cleaned_caption
0	ROCOv2_2023_train_000001	Head CT demonstrating left parotiditis.	head ct demonstrating left parotiditis
1	ROCOv2_2023_train_000002	Acquired renal cysts in end-stage renal failure: 16-year-old girl with Alport syndrome and peritoneal dialysis from the age of 2 years	acquired renal cysts in end-stage renal failure : 16 year old girl with alport syndrome and peritoneal dialysis from the age of 2 years
2	ROCOv2_2023_train_000003	Computed tomography of the chest showing the right breast nodule with irregular margins	computed tomography of the chest showing the right breast nodule with irregular margins
3	ROCOv2_2023_train_000004	Lateral view of the sacrum showing the low contrast between bone and soft tissue.	lateral view of the sacrum showing the low contrast between bone and soft tissue
4	ROCOv2_2023_train_000005	Thoracic CT scan showing perihilar pulmonary lymphadenomegaly	thoracic ct scan showing perihilar pulmonary lymphadenomegaly

Analyse Predicted Size Of Vocabulary And Excluded Tokens Based On Different Minimum Word Frequencies

Minimum word frequency is the minimum number of times a word must appear in the training captions before it is included in the vocabulary. Wu et al. (2012) analysed long-tailed clinical-term distributions of Unified Medical Language System (UMLS) terms in the Mayo Clinic clinical documents corpus. They reported that with a term frequency of 3 occurrences, 99.99% of medical terms

mentioned were retained. They also cautioned that term frequencies may not generalise directly between corpora. The tested word frequency range is therefore set from 1 to 5 for full analysis of the ROCOV2 captions.

In [12]:
```
 # set lower word-frequency threshold to include words occurring at least once
lower_threshold = 1
```

In [13]:
```
# set upper word-frequency threshold to include words occurring no more than five times
upper_threshold = 5
```

In [14]:
```
def compare_vocabulary_thresholds(training_captions, training_caption_data, minimum_threshold, maximum_threshold):

    """
    Compare vocabulary size and out-of-vocabulary (OOV) rate using different minimum word frequencies.
    The out-of-vocabulary rate is the percentage of tokens in the captions that would be excluded from the
vocabulary for the
    respective minimum word frequency.

    Args:
        training_captions (Captions): cleaned training captions
        training_caption_data (DataFrame): training captions used to calculate the OOV rate
        minimum_threshold (int): lowest minimum word-frequency
        maximum_threshold (int): highest minimum word-frequency

    Returns:
        pd.DataFrame: comparison of minimum frequency threshold, vocabulary size and OOV rate
    """

    try:

        # initialise list to store summary results for each minimum word frequency, i.e.,
        # threshold value (1-5), resulting vocabulary size and Out-Of-Vocabulary (OOV) rate (which tokens have been
excluded)
        threshold_results = []

        # for each minimum word frequency
        for threshold in range(minimum_threshold, maximum_threshold + 1):

            # instantiate new vocabulary object for the current minimum word frequency
            vocabulary_for_selected_threshold = Vocabulary()
```

```
            # build vocabulary
            vocabulary_for_selected_threshold.build_vocabulary(training_captions.captions, threshold)

            # calculate the out-of-vocabulary rate for the training captions using the selected vocabulary
threshold
            oov_for_selected_threshold =
vocabulary_for_selected_threshold.calculate_oov_statistics(training_caption_data, dataset_name="Training")

            # store the current minimum word frequency threshold, resulting vocabulary size and OOV rate
            threshold_results.append({
                "Minimum Frequency": threshold,
                "Vocabulary Size": len(vocabulary_for_selected_threshold),
                "OOV Rate (%)": oov_for_selected_threshold["oov_rate"]
            })

        # store results in DataFrame
        threshold_comparison = pd.DataFrame(threshold_results)

        # return results
        return threshold_comparison

    except Exception as e:
        raise RuntimeError(f"Exception : {str(e)}")
```

In [15]:
```
threshold_comparison = compare_vocabulary_thresholds(training_captions, training_caption_data, lower_threshold,
upper_threshold)
```

In [16]:
```
display(threshold_comparison)
```

	Minimum Frequency	Vocabulary Size	OOV Rate (%)
0	1	34026	0.000000
1	2	17602	1.281304
2	3	12892	2.016197
3	4	10621	2.547708
4	5	9187	2.995197

In [17]:

```
def plot_vocabulary_threshold_comparison(threshold_comparison):

    """
    Plot vocabulary size and out-of-vocabulary (OOV) rate across different minimum word frequencies

    Args:
        threshold_comparison (pd.DataFrame): DataFrame containing the columns 'Minimum Frequency', 'Vocabulary
Size', and 'OOV Rate (%)'
    """

    try:

        # create figure with a width of 8 inches and height of 5 inches
        fig, ax1 = plt.subplots(figsize=(8, 5))

        # plot vocabulary size
        ax1.plot(threshold_comparison["Minimum Frequency"], threshold_comparison["Vocabulary Size"], marker="o",
linewidth=2, color="tab:blue")

        # label x-axis
        ax1.set_xlabel("Minimum Word Frequency")

        # label y-axis
        ax1.set_ylabel("Vocabulary Size", color="tab:blue")

        # set the x-axis tick positions
        ax1.set_xticks(threshold_comparison["Minimum Frequency"])

        #  set y-axis tick-label colour
        ax1.tick_params(axis="y", labelcolor="tab:blue")

        # add grid lines for vidual comparison with y-axis
        ax1.grid(axis="y", linestyle="--", alpha=0.25)

        # create a secondary y-axis and plot OOV rate
        ax2 = ax1.twinx()

        # plot the OOV rate for each minimum word frequency on the secondary y-axis
        ax2.plot(threshold_comparison["Minimum Frequency"], threshold_comparison["OOV Rate (%)"], marker="s",
linewidth=2, color="tab:orange")
```

```
        # set the secondary y-axis label to show the out-of-vocabulary rate as a percentage
        ax2.set_ylabel("OOV Rate (%)", color="tab:orange")

        # set the colour of the numerical labels displayed along the secondary y-axis
        ax2.tick_params(axis="y", labelcolor="tab:orange")

        # label vocabulary size inside the graph without covering the line
        ax1.annotate("Vocabulary Size", xy=(threshold_comparison["Minimum Frequency"].iloc[-1],
                                            threshold_comparison["Vocabulary Size"].iloc[-1]), xytext=(-15, 18),
                                            textcoords="offset points", ha="right", va="bottom", color="tab:blue",
                                            fontsize=9, bbox={"facecolor": "white", "edgecolor": "none", "alpha":
0.85})

        # label OOV rate inside the graph without covering the line
        ax2.annotate("OOV Rate", xy=(threshold_comparison["Minimum Frequency"].iloc[-1],
                                     threshold_comparison["OOV Rate (%)"].iloc[-1]), xytext=(-15, -18),
                                     textcoords="offset points", ha="right", va="top", color="tab:orange",
                                     fontsize=9, bbox={"facecolor": "white", "edgecolor": "none", "alpha": 0.85})

        # chart title
        ax1.set_title("Vocabulary Size and OOV Rate by Minimum Word Frequency")

        # hide the top border of the primary plotting axis
        ax1.spines["top"].set_visible(False)

        # hide the top border of the secondary plotting axis
        ax2.spines["top"].set_visible(False)

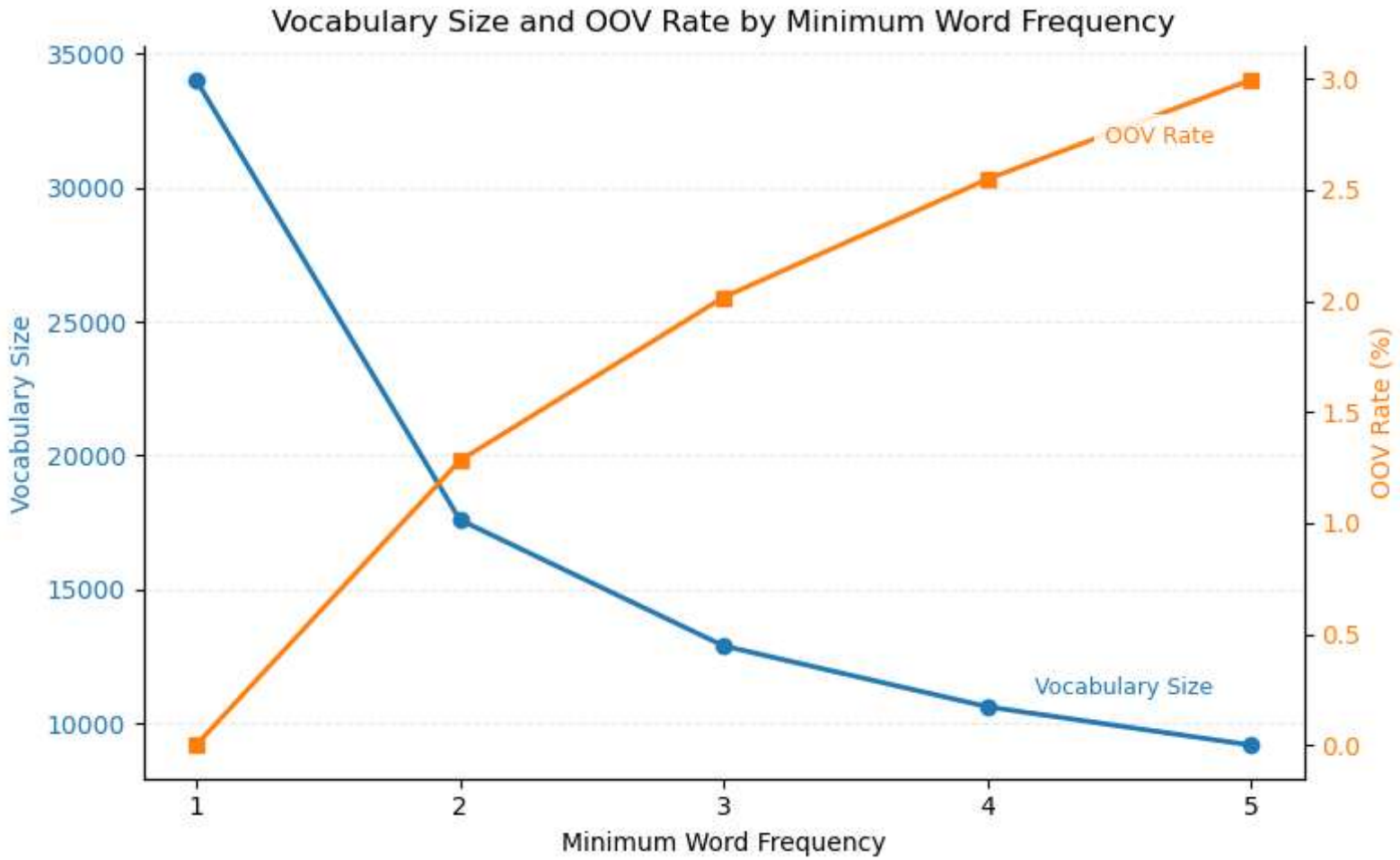
        # automatically adjust spacing between chart elements to prevent labels and titles from overlapping
        plt.tight_layout()

        # display plot
        plt.show()

    except Exception as e:
        raise RuntimeError(f"Exception : {str(e)}")
```

In [18]:

```
plot_vocabulary_threshold_comparison(threshold_comparison)
```



In [19]:
```
def plot_rank_frequencies(training_captions, lower_threshold, upper_threshold):

    """
    Plot the vocabulary rank-frequency distribution for different minimum
    word-frequency thresholds on a single graph.

    Args:
        training_captions (Captions): cleaned training captions.
        lower_threshold (int): lowest minimum word-frequency threshold.
        upper_threshold (int): highest minimum word-frequency threshold.

    Returns:
```

```
        None: displays the rank-frequency plot.
    """

    try:

        # create a range of minimum word frequencies, i.e., from the lower threshold to the upper threshold
        thresholds = range(lower_threshold, upper_threshold + 1)

        # create a single figure
        fig, ax = plt.subplots(figsize=(10, 6))

        # vertical positions used to separate the annotation labels and prevent text from overlapping
        annotation_offsets = {1: (-20, 30), 2: (70, 70), 3: (55, 30), 4: (45, 60), 5: (20, 95)}

        # plot each minimum word-frequency threshold on the same graph
        for current_minimum_frequency in thresholds:

            # instantiate a new vocabulary object for the current test threshold
            test_vocabulary = Vocabulary()

            # build test vocabulary
            test_vocabulary.build_vocabulary(training_captions.captions, current_minimum_frequency)

            # sort retained (vocabulary) word frequencies from highest to lowest
            word_frequencies = sorted(
                [
                    count
                    for word, count in test_vocabulary.word_counts.items()
                    if count >= current_minimum_frequency
                ],
                reverse=True
            )

            # create ranks from most frequent to least frequent word
            word_ranks = range(1, len(word_frequencies) + 1)

            # plot the current minimum word-frequency threshold
            ax.plot(word_ranks, word_frequencies)

            # get the final point of the current line
            final_rank = len(word_frequencies)
```

```
            # retrieve the lowest word-occurrence frequency
            final_frequency = word_frequencies[-1]

           # get horizontal and vertical annotation offsets for the current threshold
            annotation_x_offset, annotation_y_offset = annotation_offsets.get(current_minimum_frequency, (20, 20))

            # align frequency 1 to the right; all others to the left
            horizontal_alignment = "right" if current_minimum_frequency == 1 else "left"

            # add an arrow and text showing where the line ends
            ax.annotate(
                f"Frequency = {current_minimum_frequency}\n"
                f"{final_rank:,} words in dictionary",
                xy=(
                    final_rank,
                    final_frequency
                ),
                xytext=(
                    annotation_x_offset,
                    annotation_y_offset
                ),
                textcoords="offset points",
                ha=horizontal_alignment,
                va="bottom",
                arrowprops={
                    "arrowstyle": "->"
                },
                fontsize=9
            )

        # use a logarithmic scale for word frequency
        ax.set_yscale("log")

        # label x-axis
        ax.set_xlabel("Vocabulary Word Rank")

        # label y-axis
        ax.set_ylabel("Word Frequency")

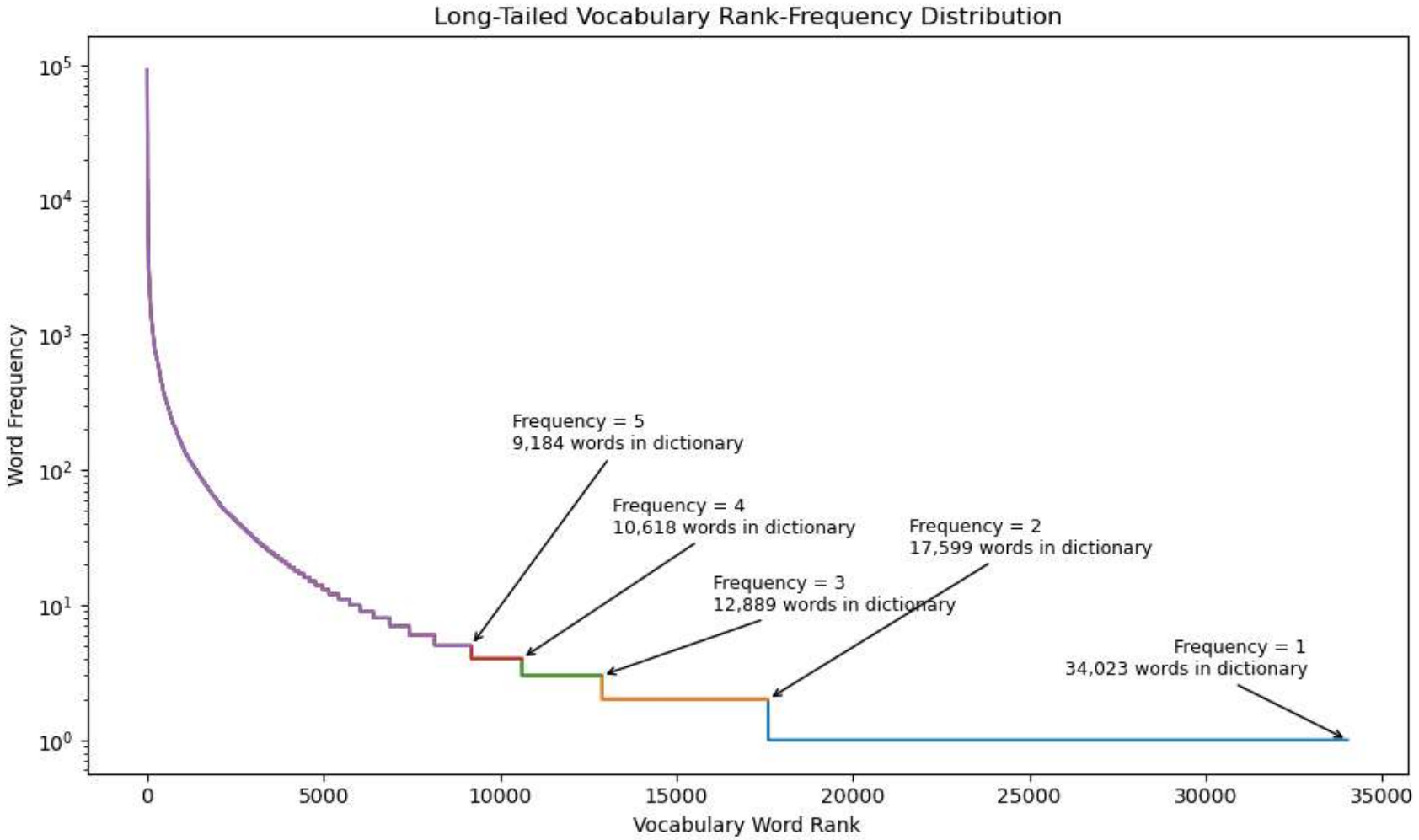
        # set title
        ax.set_title("Long-Tailed Vocabulary Rank-Frequency Distribution")
```

```
        # automatically adjust spacing between plot elements to prevent labels and titles from overlapping
        plt.tight_layout()

        # display plot
        plt.show()

    except Exception as e:
        raise RuntimeError(f"Exception : {str(e)}")
```

In [20]:
```
plot_rank_frequencies(training_captions, lower_threshold, upper_threshold)
```



The minimum word frequency for inclusion in the vocabulary will be set to 2 to reduce noise and sparsity whilst maintaining rare medical terms, i.e., balancing inclusion of medical terms and filtering of the large singleton tail :

- A minimum word frequency equalt to 2 provides the strongest initial reduction in vocabulary size, reducing the vocabulary from 34,026 to 17,602 words, a reduction of approximately 48.3%. The rank-frequency graph shows that these singleton words form a substantial part of the long tail of the vocabulary
- The resulting OOV rate is only 1.28%, indicating that singleton words account for a large proportion of the vocabulary but only a small proportion of the total training-caption tokens. Increasing the threshold further produces progressively smaller reductions in vocabulary size while continuing to increase the OOV rate. For example, increasing the threshold from 2 to 3 reduces the vocabulary by a further 4,710 words, but increases the OOV rate from 1.28% to 2.02%

In [21]:
```
minimum_word_frequency = 2
```

Build Vocabulary

In [22]:
```
# instantiate Vocabulary object
vocabulary = Vocabulary()
```

In [23]:
```
# build vocabulary, ignoring any word that appears less minimum word frequency
vocabulary.build_vocabulary(training_captions.captions, minimum_word_frequency)
```

Out[23]:
```
<vocabulary.Vocabulary at 0x12da6354b90>
```

In [24]:
```
# return count of number of unique words in the vocabulary, including <pad>, <unk> and <end>
len(vocabulary.word2idx)
```

Out[24]:
```
17602
```

In [25]:
```
# save vocabulary file, numerical Ids (mappings) as pyTorch file
vocabulary.save("ROCOv2/vocabulary.pt")
```

In [26]:
```
# confirm size of the vocabulary
len(vocabulary)
```

Out[26]:
```
17602
```

In [27]:
```
# reload the saved vocabulary file for testing
reloaded_vocabulary = Vocabulary.load("ROCOv2/vocabulary.pt")
```

In [28]:
```
# confirm size of the reloaded vocabulary, includung special characters
len(reloaded_vocabulary)
```

Out[28]:
```
17602
```

In [29]:
```
# confirm mappings and settings were preserved
assert reloaded_vocabulary.word2idx == vocabulary.word2idx
assert reloaded_vocabulary.idx2word == vocabulary.idx2word
assert reloaded_vocabulary.minimum_word_frequency == vocabulary.minimum_word_frequency
assert reloaded_vocabulary.idx == vocabulary.idx
```

Additional Out-Of-Vocabulary Diagnostics

In [30]:
```
# calculate the out-of-vocabulary statistics for the training captions against the final vocabulary
training_oov = vocabulary.calculate_oov_statistics(training_caption_data, dataset_name="Training")
```

In [31]:
```
def plot_oov_statistics(oov_statistics):
    """
    Display token-level and caption-level out-of-vocabulary (OOV) statistics
    as horizontal percentage-based stacked bar charts with summary tables.

    Args:
        oov_statistics (dict): OOV statistics returned by calculate_oov_statistics()
    """
    try:

        # retrieve dataset name
        dataset_name = oov_statistics["dataset"]

        # retrieve total number of tokens
        total_tokens = oov_statistics["total_tokens"]

        # retrieve total number of out-of-vocabulary tokens
        oov_tokens = oov_statistics["oov_tokens"]

        # retrieve precentage of tokens out-of-vocabulary
```

```
        token_oov_percentage = oov_statistics["oov_rate"]

        # calculate percentage of tokens that are present in the vocabulary
        in_vocabulary_percentage = 100 - token_oov_percentage

        # retrieve total number of captions
        total_captions = oov_statistics["total_captions"]

        # retrieve number of captions containing at least one out-of-vocabulary token
        captions_with_oov = oov_statistics["captions_with_oov"]

        # retrieve percentage of captions containing at least one out-of-vocabulary token
        caption_oov_percentage = oov_statistics["oov_caption_percentage"]

        # calculate percentage of captions containing no OOV words
        captions_without_oov_percentage = 100 - caption_oov_percentage

        # create vertically stacked charts
        fig, axes = plt.subplots(2, 1, figsize=(10, 5))

        # plot the percentage of tokens that are present in the vocabulary
        axes[0].barh(0, in_vocabulary_percentage, height=0.25, color="black", edgecolor="black", linewidth=0.8)

        # plot the percentage of out-of-vocabulary tokens
        axes[0].barh(0, token_oov_percentage, left=in_vocabulary_percentage, height=0.25, color="lightgrey",
edgecolor="grey", linewidth=0.8, hatch="//")

        # set token-level chart title
        axes[0].set_title(f'{dataset_name} Token-Level OOV', loc="left", pad=10)

        # display token category labels centred above each bar segment
        axes[0].text(in_vocabulary_percentage / 2, 0.22, "In Vocabulary", ha="center", va="bottom")
        axes[0].text(in_vocabulary_percentage + (token_oov_percentage / 2), 0.22, "OOV", ha="center", va="bottom")

        # create token-level summary table
        token_table = axes[0].table(cellText=[[f"{total_tokens:,}", f"{oov_tokens:,}", f"
{token_oov_percentage:.2f}%"]],
                                    colLabels=["Total Tokens", "OOV Tokens", "OOV Rate (%)"], cellLoc="center",
loc="bottom",
                                    bbox=[0, -0.42, 1, 0.30])

        # set token-level table borders
```

```
        for cell in token_table.get_celld().values():
            cell.set_linewidth(0.8)

        # plot the percentage of captions containing no OOV words
        axes[1].barh(0, captions_without_oov_percentage, height=0.25, color="black", edgecolor="black",
linewidth=0.8)

        # plot the percentage of captions containing at least one OOV word
        axes[1].barh(0, caption_oov_percentage, left=captions_without_oov_percentage, height=0.25,
                     color="lightgrey", edgecolor="grey", linewidth=0.8, hatch="//")

        # set caption-level chart title
        axes[1].set_title(f'{dataset_name} Caption-Level OOV', loc="left", pad=10)

        # display caption category labels centred above each bar segment
        axes[1].text(captions_without_oov_percentage / 2, 0.22, "Without OOV", ha="center", va="bottom")
        axes[1].text(captions_without_oov_percentage + (caption_oov_percentage / 2), 0.22, "With OOV", ha="center",
va="bottom")

        # create caption-level summary table
        caption_table = axes[1].table(cellText=[[f"{total_captions:,}", f"{captions_with_oov:,}", f"
{caption_oov_percentage:.2f}%"]],
                                      colLabels=["Total Captions", "Captions With OOV Word", "OOV Caption (%)"],
cellLoc="center", loc="bottom",
                                      bbox=[0, -0.42, 1, 0.30])

        # set caption-level table borders
        for cell in caption_table.get_celld().values():
            cell.set_linewidth(0.8)

        # remove axes, tick marks and borders
        for axis in axes:

            # set percentage scale
            axis.set_xlim(0, 100)

            # set vertical spacing
            axis.set_ylim(-0.45, 0.55)

            # remove axis tick marks
            axis.set_xticks([])
            axis.set_yticks([])
```

```
            # remove chart borders
            for spine in axis.spines.values():
                spine.set_visible(False)

        # reduce spacing between charts
        plt.subplots_adjust(hspace=0.75)

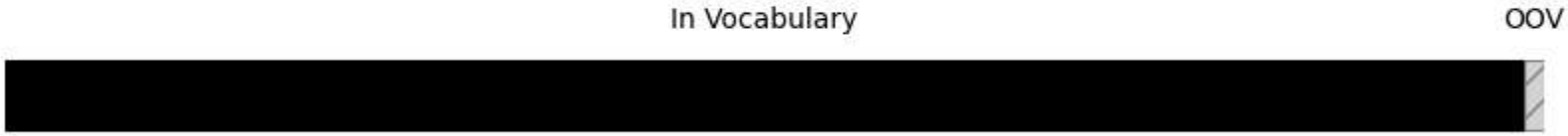
        # display charts
        plt.show()

    except Exception as e:
        raise RuntimeError(f"Exception : {str(e)}")
```

In [32]:

```
# display OOV statistics graphically
plot_oov_statistics(training_oov)
```

Training Token-Level OOV



Total Tokens	OOV Tokens	OOV Rate (%)
1,281,819	16,424	1.28%

Training Caption-Level OOV



Total Captions	Captions With OOV Word	OOV Caption (%)
59,958	11,731	19.57%

Medical Term Diagnostics

The U.S. National Library of Medicine (NLM) provides biomedical information organisation and maintains MeSH (Medical Subject Headings), a controlled vocabulary used to describe and organise biomedical and health-related information (NLM, 2026).

MeSH XML files:

- desc2026.xml — contains the main MeSH Descriptor records, including recognised biomedical concepts and their associated terms

- supp2026.xml — contains Supplementary Concept Records, which extend MeSH coverage with additional biomedical concepts such as chemicals, drugs and other specialised terms

These files act as an external biomedical terminology source which will be used to highlight the exclusion of recognised medical or biomedical terminology.

In [33]:
```
def load_mesh_terms(mesh_filenames):
    """
    Extract medical terms from MeSH XML files

    args:
        mesh_filenames (list): MeSH XML file paths

    returns:
        set: unique MeSH medical terms
    """
    try:

        # initialise empty set to store unique MeSH terms
        mesh_terms = set()

        # loop through each MeSH XML file
        for mesh_filename in mesh_filenames:

            # parse the XML file
            root = ET.parse(mesh_filename).getroot()

            # loop through each MeSH term
            for term in root.findall(".//Term/String"):

                # check that the term contains text
                if term.text:

                    # add the MeSH term to the set
                    mesh_terms.add(term.text)

        # return unique MeSH terms
        return mesh_terms
```

```
    except Exception as e:
        raise RuntimeError(f"Exception : {str(e)}")
```

In [34]:
```
# define MeSH XML files
mesh_filenames = ["NLM/desc2026.xml", "NLM/supp2026.xml"]
```

In [35]:
```
# extract unique MeSH terms
mesh_terms = load_mesh_terms(mesh_filenames)
```

In [36]:
```
# display number of unique MeSH terms
print(f"Unique MeSH Terms: {len(mesh_terms):,}")
```

```
Unique MeSH Terms: 996,029
```

In [37]:
```
def create_mesh_word_set(mesh_terms):
    """
    Clean and normalise MeSH terms using the same preprocessing rules as the ROCOv2 captions
    and convert them into a set of individual words.

    Args:
        mesh_terms (set): unique MeSH medical terms

    Returns:
        set: unique cleaned individual MeSH words
    """
    try:

        # create a Captions object for MeSH term preprocessing
        mesh_captions = Captions()

        # convert MeSH terms to a DataFrame using the Caption column required by the Captions class
        mesh_captions.captions = pd.DataFrame({"Caption": list(mesh_terms)})

        # clean and normalise MeSH terms using the same processing as the ROCOv2 captions
        mesh_captions.clean_captions()

        # initialise empty set to store unique MeSH words
        mesh_words = set()

        # loop through each cleaned MeSH term
        for term in mesh_captions.captions["cleaned_caption"].dropna():
```

```
            # split the cleaned MeSH term into individual words
            words = term.split()

            # add individual words to the MeSH word set
            mesh_words.update(words)

        # remove empty word if present
        mesh_words.discard("")

        # return unique cleaned MeSH words
        return mesh_words

    except Exception as e:
        raise RuntimeError(f"Exception : {str(e)}")
```

In [38]:
```
# clean, normalise and convert MeSH terms into individual words
mesh_words = create_mesh_word_set(mesh_terms)
```

In [39]:
```
# display number of unique MeSH words
print(f"Unique MeSH Words: {len(mesh_words):,}")
```

```
Unique MeSH Words: 568,727
```

In [40]:
```
# identify excluded vocabulary words and excluded words matching MeSH terminology
excluded_word_counts, excluded_medical_word_counts = vocabulary.identify_excluded_medical_words(mesh_words)
```

In [41]:
```
def calculate_excluded_medical_statistics(excluded_word_counts, excluded_medical_word_counts):
    """
    Calculate summary statistics for vocabulary words excluded by the minimum-frequency threshold

    Args:
        excluded_word_counts (Counter): excluded vocabulary words and their training frequencies
        excluded_medical_word_counts (Counter): excluded vocabulary words present in the MeSH word set

    Returns:
        dict: excluded medical-word summary statistics
    """
    try:

        # calculate total number of unique words excluded from the vocabulary
```

```
        total_excluded_words = len(excluded_word_counts)

        # calculate number of unique excluded words present in the MeSH word set
        excluded_medical_words = len(excluded_medical_word_counts)

        # calculate percentage of excluded words present in the MeSH word set
        excluded_medical_percentage = ((excluded_medical_words / total_excluded_words) * 100 if
total_excluded_words > 0 else 0.0)

        # return excluded medical-word summary statistics
        return {
            "total_excluded_words": total_excluded_words,
            "excluded_medical_words": excluded_medical_words,
            "excluded_medical_percentage": excluded_medical_percentage,
        }

    except Exception as e:
        raise RuntimeError(f"Exception : {str(e)}")
```

In [42]:
```
# calculate summary statistics for excluded medical words
excluded_medical_statistics = calculate_excluded_medical_statistics(excluded_word_counts,
excluded_medical_word_counts)
```

In [43]:
```
def plot_excluded_medical_statistics(excluded_medical_statistics, excluded_medical_word_counts,
dataset_name="Training"):
    """
    Display excluded vocabulary words and MeSH medical-term matches
    as a horizontal percentage-based stacked bar chart with summary tables.

    Args:
        excluded_medical_statistics (dict): excluded medical-word summary statistics
        excluded_medical_word_counts (Counter): excluded vocabulary words present in the MeSH word set
        dataset_name (str): display name for charts / graphs
    """
    try:

        # retrieve total number of unique words excluded from the vocabulary
        total_excluded_words = excluded_medical_statistics["total_excluded_words"]

        # retrieve number of excluded words present in the MeSH word set
        excluded_medical_words = excluded_medical_statistics["excluded_medical_words"]
```

```
        # retrieve percentage of excluded words present in the MeSH word set
        excluded_medical_percentage = excluded_medical_statistics["excluded_medical_percentage"]

        # calculate number of excluded words not present in the MeSH word set
        excluded_non_medical_words = total_excluded_words - excluded_medical_words

        # calculate percentage of excluded words not present in the MeSH word set
        excluded_non_medical_percentage = 100 - excluded_medical_percentage

        # create chart
        fig, axis = plt.subplots(1, 1, figsize=(10, 2.5))

        # plot the percentage of excluded words not present in the MeSH word set
        axis.barh(0, excluded_non_medical_percentage, height=0.25, color="black", edgecolor="black", linewidth=0.8)

        # plot the percentage of excluded words present in the MeSH word set
        axis.barh(0, excluded_medical_percentage, left=excluded_non_medical_percentage, height=0.25,
                  color="lightgrey", edgecolor="grey", linewidth=0.8, hatch="//")

        # set chart title
        axis.set_title(f'{dataset_name} Excluded Medical-Term Diagnostics', loc="left", pad=10)

        # display category labels centred above each bar segment
        axis.text(excluded_non_medical_percentage / 2, 0.22, "Not Matched to MeSH", ha="center", va="bottom")
        axis.text(excluded_non_medical_percentage + (excluded_medical_percentage / 2), 0.22, "Matched to MeSH",
ha="center", va="bottom")

        # create excluded medical-word summary table
        summary_table = axis.table(cellText=[[f"{total_excluded_words:,}", f"{excluded_medical_words:,}",
                                              f"{excluded_medical_percentage:.2f}%"]], colLabels=["Excluded Words",
"Excluded Medical Words",
                                                                                                  "Excluded Medical
Words (%)"],

cellLoc="center", loc="bottom",
                                   bbox=[0, -0.45, 1, 0.30])

        # set table borders
        for cell in summary_table.get_celld().values():
            cell.set_linewidth(0.8)
```

```
        # set percentage scale
        axis.set_xlim(0, 100)

        # set vertical spacing
        axis.set_ylim(-0.45, 0.55)

        # remove axis tick marks
        axis.set_xticks([])
        axis.set_yticks([])

        # remove chart borders
        for spine in axis.spines.values():
            spine.set_visible(False)

        # chart alignment
        plt.tight_layout()

        # display chart
        plt.show()

        # create table of excluded words matched to MeSH terminology
        excluded_medical_words_table = pd.DataFrame(excluded_medical_word_counts.most_common(), columns=["Excluded
Medical Word", "Training Frequency"])

        # display excluded medical words
        display(excluded_medical_words_table)

    except Exception as e:
        raise RuntimeError(f"Exception : {str(e)}")
```

In [44]:

```
# display excluded medical-word statistics
plot_excluded_medical_statistics(excluded_medical_statistics, excluded_medical_word_counts,
dataset_name="Training")
```

Training Excluded Medical-Term Diagnostics



Excluded Words	Excluded Medical Words	Excluded Medical Words (%)
16,424	3,629	22.10%

	Excluded Medical Word	Training Frequency
0	parotiditis	1
1	alport	1
2	cristae	1
3	strawberry	1
4	faced	1
...	...	...
3624	jugulare	1
3625	undermodeling	1
3626	traits	1
3627	dc1	1
3628	opercular	1

3629 rows × 2 columns

Issues That Could Increase The Risk Of Hallucinations

- **The training vocabulary cannot represent unseen terminology.** A medically correct term that appears only in unseen data cannot be generated as that exact vocabulary item, increasing the risk of omission or substitution.

- **Training captions describe facts that are contextual and not visually inferable from the image, e.g., the patient is 16 years old.** The model may learn to generate clinical context or patient history that is not supported by the image itself.

- **A whole-word vocabulary limits rare medical terminology, i.e., captions are split using whitespace and each resulting word is added to the vocabulary as an independent entry.** A whole-word vocabulary is potentially problematic for medical captioning where rare and morphologically complex terms are common. For example parotiditis and parotid. There is no subword or character-level mechanism for constructing rare specialist terminology from smaller reusable components.

- **Rare medical terms excluded become <unk>.** For example,3,629 excluded singleton word forms match MeSH terminology. Clinically meaningful terms are therefore excluded from the vocabulary, increasing the chance that the model generates a more frequent but incorrect medical term.

- **All unseen or out-of-vocabulary words collapse to the same <unk> representation.** Unseen concepts may be replaced by plausible but incorrect known vocabulary words.

- **Trailing hyphens are removed from words.** Clinically meaningful minus symbols or symbols indicating a negative state could be removed potentially altering the meaning of the target caption.

- **The allowed-character filter removes potentially meaningful relation or uncertainty symbols, such as =, ±, <, >, and arrows.** This could alter clinical comparisons, relationships, or expressions of uncertainty.

- **Numbers and measurement units or symbols are separated into independent tokens.** The relationship between a numerical value and its unit may be weakened increasing the possibility of incorrect value-unit combinations.

- **All text is lower-cased.** Case-sensitive biomedical abbreviations or identifiers may become ambiguous after normalisation.

- **Spelling errors can survive caption cleaning.** Misspellings may become separate vocabulary entries

References

Cruz Diaz, N.P. and Mana Lopez, M, 'An analysis of biomedical tokenization: problems and strategies', Proceedings of the Sixth International Workshop on Health Text Mining and Information Analysis', pages 40-49 (2015). Available At: https://aclanthology.org/W15-2605/

He, B., Guan, Y., Cheng, J., Cen, K. and Hua, W. CRFs Based De-identification Of Medical Records. Science Direct, 2015. Available At: https://doi.org/10.1016/j.jbi.2015.08.012, pages 39-46

National Library of Medicine, MeSH XML data files (2026). Available At: https://nlmpubs.nlm.nih.gov/projects/mesh/MESH_FILES/xmlmesh/

Moon, S.R., Pakhomov, S., Ryan, J. and Melton, G.B., 'Automated non-alphanumeric symbol resolution in clinical texts', AMIA Annual Symposium Proceedings (2011) pages 979–986. Available At : https://pmc.ncbi.nlm.nih.gov/articles/PMC3243158/

OCOM5100M Programming For Data Science

OCOM5101M Data Science

OCOM5200M Machine Learning

OCOM5203M Deep Learning

OCOM5203M Deep Learning Module, Summative Assessment 2

OCOM5204M Data Mining And Analytics

Project Jupyter, Jupyter Notebook documentation. Available At: https://jupyter-notebook.readthedocs.io/en/stable/

Python Software Foundation, Python 3.13 documentation (2026). Available At: https://docs.python.org/3.13/

Python Software Foundation, re — Regular expression operations (2026). Available At: https://docs.python.org/3/library/re.html

ROCOv2: Radiology Objects in Context Version 2, An Updated Multimodal Image Dataset. Available At : https://zenodo.org/records/10821435

Rodríguez, G, Pythex: a Python regular expression editor (n.d). Available At: https://pythex.org/

Wu, S.T., Liu, H., Li, D., Tao, C., Musen, M.A., Chute, C.G. and Shah, N.H. , 'Unified Medical Language System term occurrences in clinical notes: a large-scale corpus analysis', Journal of the American Medical Informatics Association, (2012) pages e149–e156. Available At : https://academic.oup.com/jamia/article/19/e1/e149/2909195?login=false